Genetic Programming Hyper-Heuristics for the Job Shop Scheduling Problem with Interval Durations: Robustness-Aware Interpretable Rules that Generalize Across Instance Sizes

Hernán Díaz

Department of Computing, University of Oviedo, Campus de Gijón, Gijón, 33204, Spain

Abstract

The automated design of dispatching rules has not previously been explored for the job shop scheduling problem with interval durations (IJSP), where each processing time is known only to lie within an interval and schedules are built and compared by interval arithmetic. We present a genetic programming hyper-heuristic for this problem: priority rules are evolved over an *interval-aware* terminal set that exposes both the worst-case values and the widths of the uncertain quantities, and are deployed in a single constructive pass. On the 70 interval Taillard instances, 30 independent evolutions average a relative error of 18.99% ($\pm$1.33) and the best rule attains 17.71%, improving on the strongest constructive baseline, Giffler–Thompson with MWKR tie-breaking at 29.5%, on every instance. Evolved on four 20$\times$15 instances, the rules transfer zero-shot to all seven size classes and to a second benchmark of classical instances, keeping their margin over the constructive baselines, and remain compact, human-readable expressions. Controlled ablations then delimit what interval awareness contributes. For expected

Email address: diazhernan@uniovi.es (Hernán Díaz)

makespan it is unnecessary: removing the width terminals, or evolving on the crisp midpoint instances with scalar fitness, does equally well. Its value lies in robustness: an objective that penalizes interval width, one with no crisp counterpart, yields substantially narrower predicted intervals and a deviation of the executed makespan from its prediction about one fifth lower, neither attainable without the width terminals. Both gains are paid for in expected makespan, with the weight of the width penalty setting the trade-off.
Keywords: interval job shop scheduling, genetic programming, hyper-heuristics, dispatching rules, scheduling under uncertainty, robustness

1. Introduction

Real manufacturing environments rarely know the exact duration of an operation in advance [1]. The interval job shop scheduling problem (IJSP) models this uncertainty in a minimal, distribution-free way: each processing time is only assumed to lie within a known interval, and schedules are built and evaluated with interval arithmetic, so that the makespan of a solution is itself an interval and solutions are compared by an interval ranking. Population metaheuristics for the IJSP, among them genetic algorithms and artificial bee colony variants, achieve strong results at the cost of building and evaluating large numbers of complete solutions for every instance [2, 3, 4].

At the opposite end of the computational spectrum are *dispatching rules*: priority functions that build a schedule in a single constructive pass by repeatedly selecting the next operation to start. They combine speed, interpretability and ease of deployment. However, hand-crafted rules leave a large quality gap with respect to search-based methods, as the experiments

reported later in this paper confirm.

For *deterministic* scheduling, evolving rules automatically with genetic programming (GP) has been shown to narrow this gap substantially [5, 6]. For scheduling under interval uncertainty, however, the automated design of dispatching rules remains, to the best of our knowledge, unexplored. Moving automated rule design to interval durations raises questions that do not arise in the deterministic case: which attributes the evolution actually selects, when the quantities describing a candidate operation (its duration, its earliest start, the work remaining in its job) are intervals rather than numbers; whether the *magnitude* of the uncertainty carries information that is useful for dispatching, or whether it is the position of the intervals that drives the decision; and whether an objective that rewards a narrow makespan interval, rather than a low one, leads the evolution to schedules whose execution departs less from what was predicted. Answering these questions empirically requires exposing uncertainty to the rule as first-class attributes, which is the starting point of this work. A further question, how interval makespans should be compared at all when they admit no natural total order, we do not reopen: we adopt the ranking that a systematic study of the alternatives found most robust [2].

This paper makes the following contributions:

1. A GP hyper-heuristic that brings automated rule design to the IJSP: priority expressions over a terminal set in which the duration, earliest start and work remaining of a candidate operation appear both as worst-case values and as widths, with fitness computed by interval arithmetic. The rules it produces are compact enough to be simplified and read by hand (Sections 4 and 7.1).

2. An empirical study on the 70 interval Taillard instances showing that evolved rules (i) outperform every deterministic constructive baseline by a wide margin at equal cost, (ii) generalize zero-shot across instance sizes (evolved on 20×15, best class 50×15), and (iii) transfer to a second benchmark of classical instances, keeping the same margin over the constructive baselines (Section 6).
3. A delimitation of what interval awareness contributes, in two halves. It contributes nothing to expected makespan: removing the width terminals leaves it unchanged, and so does replacing the interval-arithmetic fitness by scalar evolution on the crisp midpoint instances (Section 7.2). Its value is robustness: an objective that penalizes interval width, which has no crisp counterpart, cuts the deviation of the executed makespan from its prediction by about a fifth, a reduction unattainable without the width terminals and paid for in expected makespan, with the weight of the penalty tracing the frontier between the two (Section 7.4).

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 formalizes the IJSP. Section 4 presents the GP hyper-heuristic, from both the hyper-heuristic and the evolutionary viewpoints. Section 5 details the benchmark, the training protocol and the irace configuration study. Section 6 reports the empirical results, and Section 7 analyzes them: rule anatomy, the interval-awareness ablation, executional robustness and limitations. Section 8 concludes.

2. Related Work

2.1. Job shop scheduling under interval uncertainty

Interval durations offer a distribution-free uncertainty model requiring only bounds, in contrast to stochastic and fuzzy approaches [7]: they are the natural choice when historical data are insufficient to estimate distributions and a decision maker can only supply a minimum and a maximum duration. Scheduling with interval processing times has been studied across machine environments, from the single machine [8] to the flexible job shop [9], with the flow shop and others surveyed in [1]; recent work extends the model to richer variants such as assembly [10] and energy-aware flexible [11] shops. It has also been studied analytically, through measures of the problem uncertainty induced by the intervals and of the stability of optimal sequences [12], and through the execution of schedules built under uncertain durations [13], the question Section 7.4 examines for evolved rules.

For the job shop with makespan minimization, which concerns us here, the literature is dominated by population-based metaheuristics. Lei's population-based neighborhood search [14], which ranks interval makespans by a possibility degree, was an early reference point; it was subsequently outperformed by a genetic algorithm with a systematic study of interval rankings [2] and by artificial bee colony variants [3, 4], which report the best published results for this problem. Related objectives have also been addressed under interval uncertainty, notably tardiness minimization [15]. All of these are iterative search methods: they build and evaluate many complete solutions and refine them over many iterations for each instance solved. Fast constructive methods for the IJSP have received far less attention, and dispatching rules with interval-aware lexicographic comparisons

are the point of comparison they are measured against, the role they play in this paper (Section 6.3).

2.2. Dispatching rules for job shop scheduling

Priority dispatching rules are among the oldest and most widely deployed scheduling heuristics. Classical rules score each ready operation by a single attribute, and we adopt their standard acronyms throughout this paper. *Shortest* and *longest processing time* (SPT, LPT) look at the duration of the operation itself. *Earliest starting time* (EST) prefers the operation that can begin soonest given the current state of its job and its machine. *Most work remaining* (MWKR) and *most operations remaining* (MOR) look instead at the job the operation belongs to, and favor the one with the largest amount of pending work, measured respectively in time units and in number of operations. The *critical ratio* (CR) weighs the work still to be done against the time available until a due date. Surveys catalogue more than a hundred such variants [16, 17], and more recent ones compare those in current use across machine environments [18]. A step above single-attribute rules, the algorithm of Giffler and Thompson (G&T) [19] restricts each choice to a machine's conflict set, guaranteeing *active* schedules and markedly improving plain rules when a priority criterion is used to break ties inside that set; we write G&T-SPT and G&T-MWKR for the resulting combinations. Stronger constructive procedures exist for the *deterministic* job shop, notably the shifting bottleneck heuristic [20], which repeatedly solves one-machine subproblems with heads and tails by means of Carlier's algorithm [21]. These fall outside the scope of the present work: their machinery rests on exact one-machine optimization and critical-path arguments that do not extend directly to interval arithmetic, where no natural total order

is available. Hand-crafted rules, however, plateaued: no fixed combination of attributes performs well across shop configurations and objectives [17], which motivated their *automated* design.

2.3. Automated design of dispatching rules

Genetic programming [22, 23] evolves a population of expressions, usually represented as trees, by applying selection and variation operators directly on their structure. GP hyper-heuristics apply this machinery to *priority expressions* over scheduling attributes, and constitute the standard approach to automated rule design for production scheduling [5, 6]. Evolved rules have been shown to outperform hand-crafted ones across job shop variants, with active research on surrogate fitness [24], feature selection [25] and multi-objective formulations [26]. Beyond single rules, a further line combines several evolved rules into *ensembles* that jointly construct or vote on a solution [27], and recent work explores alternatives to evolutionary search for the same design task, namely systematic tree search over the space of expression trees [28], reporting rules of comparable quality but smaller size and better readability. All of this work targets deterministic (or dynamic-stochastic) settings. Closest to our setting, Karunakaran et al. [29] evolve rules for dynamic job shops with uncertain processing times, exposing summary statistics of observed deviations (an exponential moving average) to the rule; uncertainty there is realized stochastically during simulation, and rules are evaluated on sampled scenarios. A recent survey of learned optimization for the job shop [30], covering both GP and DRL constructive approaches, records no work on interval-valued processing times. To the best of our knowledge, evolving rules whose *inputs* are interval attributes, and whose fitness is computed by interval arithmetic rather than by sampling

realizations, has not been explored before.

Finally, deep reinforcement learning provides an alternative family of learned constructive methods for the JSP [31, 32]; GP rules and DRL policies are increasingly studied as competing learning paradigms for the same construction task, and controlled comparisons between them under common protocols have been identified as a gap in the recent literature [30]. Both families occupy the same computational class at deployment: a single constructive pass.

3. The Interval Job Shop Scheduling Problem

An IJSP instance consists of n jobs and m machines. Job j is a chain of m operations $o_{j1} \prec o_{j2} \prec \cdots \prec o_{jm}$, each requiring a distinct machine $\mu(o_{jl})$ exclusively and non-preemptively. The duration of operation o is an interval $\mathbf{p}_o = [\underline{p}_o, \overline{p}_o]$ with $0 < \underline{p}_o \leq \overline{p}_o$: the only assumption is that the realized duration lies within these bounds. As in this definition, boldface denotes an interval-valued quantity throughout, and the underlined and overlined symbols its lower and upper bounds.

Schedule construction uses interval arithmetic. For intervals $\mathbf{a} = [\underline{a}, \overline{a}]$ and $\mathbf{b} = [\underline{b}, \overline{b}]$, the two operations required by makespan minimization are

$$\mathbf{a} + \mathbf{b} = [\underline{a} + \underline{b},\ \overline{a} + \overline{b}], \tag{1}$$

$$\max(\mathbf{a}, \mathbf{b}) = [\max(\underline{a}, \underline{b}),\ \max(\overline{a}, \overline{b})], \tag{2}$$

both component-wise, the standard operations of interval arithmetic [33] as used in the IJSP literature [2]. A solution determines a task processing order π (a permutation with repetition of jobs, decoded left to right). Following the notation of [2, 4], let $\mathrm{PM}_o(\pi)$ and PJ_o denote the predecessors of task o

in its machine (according to π) and in its job. The *semi-active* schedule [34] starts each task at

$$\mathbf{s}_o(\pi) = \max\big(\mathbf{c}_{\mathrm{PJ}_o}(\pi),\, \mathbf{c}_{\mathrm{PM}_{o(\pi)}}(\pi)\big), \qquad \mathbf{c}_o(\pi) = \mathbf{s}_o(\pi) + \mathbf{p}_o, \tag{3}$$

so starting and completion times of all tasks are themselves intervals. The problem can be stated as

$$\min_{R} \mathbf{C}_{\max} \tag{4}$$

$$\text{s.t.} \quad \mathbf{C}_{\max} = \max_{1\le j\le n}\{\mathbf{c}_{o_{jm}}\}, \tag{5}$$

$$\underline{s}_{o_{jl}} \ge \underline{c}_{o_{j,l-1}}, \quad \overline{s}_{o_{jl}} \ge \overline{c}_{o_{j,l-1}}, \qquad 2 \le l \le m,\ 1 \le j \le n, \tag{6}$$

$$\underline{s}_o \ge \underline{c}_{o'} \ \vee\ \underline{s}_{o'} \ge \underline{c}_o, \quad \overline{s}_o \ge \overline{c}_{o'} \ \vee\ \overline{s}_{o'} \ge \overline{c}_o, \quad \forall o \ne o' : \mu(o) = \mu(o'), \tag{7}$$

where Eq. (5) defines the interval makespan as the component-wise maximum of the job completion times, Eq. (6) enforces precedence within each job and Eq. (7) forbids overlaps on each machine, in both interval components [2, 4]. The minimum in (4) is taken with respect to an interval ranking R, since intervals admit no natural total order and any ranking embodies a modeling choice [35]. We rank schedules lexicographically, comparing upper bounds first and using the lower bound only to break ties:

$$\mathbf{a} \preceq \mathbf{b} \iff \overline{a} < \overline{b} \ \vee\ \big(\overline{a} = \overline{b} \wedge \underline{a} \le \underline{b}\big), \tag{8}$$

one of the standard rankings of the IJSP literature and the one found in [2] to yield the most robust schedules. All methods in this paper, the GP rules and every baseline, share one implementation of this decoder and evaluator, so reported differences cannot stem from evaluator discrepancies.

Performance is reported as the relative error of the expected makespan with respect to a crisp reference bound LB of each instance,

$$\mathrm{RE} = \frac{E[\mathbf{C}_{\max}] - \mathrm{LB}}{\mathrm{LB}} \times 100, \qquad E[\mathbf{C}_{\max}] = \frac{\underline{C}_{\max} + \overline{C}_{\max}}{2}, \tag{9}$$

where $E[\mathbf{C}_{\max}]$ is the expected value of the interval makespan under the uniform distribution on $[\underline{C}_{\max}, \overline{C}_{\max}]$, and LB is a bound of the underlying deterministic instance: Taillard's bounds [36] for the Taillard-derived instances, the best known ones [3] for the classical set. It serves as an indicative common reference, not as a bound for the interval problem, for which none is available.

A schedule is eventually *executed* under one realization of the durations, and a second quantity of interest is how well its a-priori interval predicts that execution. Measuring the gap between a predicted schedule and its execution is the classical notion of schedule robustness [37], here specialized to interval predictions. Following [2], the $\bar{\varepsilon}$ robustness of a schedule with a fixed processing order is the average relative deviation of its executed makespan from the predicted expected value, over K realizations of the durations:

$$\bar{\varepsilon} = \frac{1}{K} \sum_{k=1}^{K} \frac{\left| C_{\max}^{\mathrm{ex},k} - E[\mathbf{C}_{\max}] \right|}{E[\mathbf{C}_{\max}]}, \tag{10}$$

where $C_{\max}^{\mathrm{ex},k}$ is the crisp makespan obtained by executing that same order when the durations turn out to take the values of realization k. A low $\bar{\varepsilon}$ therefore means that the executed makespan stays close to its a-priori expectation.

4. GP Hyper-Heuristic for the IJSP

We describe the method from two complementary viewpoints: as a *hyper-heuristic*, that is, what an individual is and how it turns an IJSP instance into a schedule (Section 4.1); and as an *evolutionary algorithm*, that is, how the population of rules is initialized, varied and selected (Section 4.2).

Table 1: Interval-aware terminal set, with exact definitions. For an eligible operation o of job j on machine $\mu(o)$: $\mathbf{p}_o = [\underline{p}_o, \overline{p}_o]$ is its duration interval; $\mathbf{C}_j$ and $\mathbf{C}_{\mu(o)}$ are the interval completion times of the last scheduled operation of job j and of machine $\mu(o)$; $\mathcal{R}_o$ is the set of operations of j after o; $\mathcal{E}$ is the current eligible set. Terminals are evaluated on *raw* (unnormalized) values.

Terminal	Definition	Role
PT	$\overline{p}_o$	worst-case processing time
PTW	$\overline{p}_o - \underline{p}_o$	duration uncertainty
EST	$\overline{e}_o = \max(\overline{C}_j, \overline{C}_{\mu(o)})$	worst-case earliest start
ESTW	$\overline{e}_o - \underline{e}_o$, with $\underline{e}_o = \max(\underline{C}_j, \underline{C}_{\mu(o)})$	start uncertainty
WKR	$\sum_{q \in \mathcal{R}_o} \overline{p}_q$	worst-case work remaining
WKRW	$\sum_{q \in \mathcal{R}_o} (\overline{p}_q - \underline{p}_q)$	remaining-work uncertainty
NOR	$\lvert\mathcal{R}_o\rvert$	operations remaining
SLACK	$\overline{e}_o - \min_{o' \in \mathcal{E}} \overline{e}_{o'}$	start slack within eligible set
ONE	1	constant

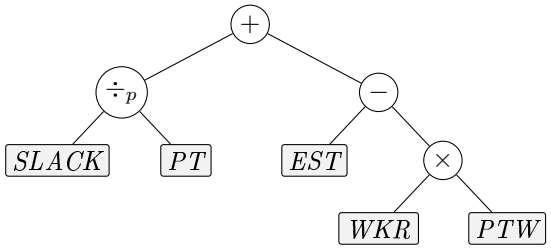

Figure 1: An individual is a priority expression tree; leaves are interval-aware attributes of the candidate operation (here the illustrative rule $SLACK/PT + EST - WKR \cdot PTW$). At each decision point the tree is evaluated on every eligible operation and the lowest-priority one is dispatched.

4.1. Hyper-heuristic view: from expression trees to schedules

An individual is a priority expression tree (Figure 1). Leaves are drawn from the interval-aware terminal set of Table 1, and internal nodes from the

function set

$$\mathcal{F} = \{ +,\ -,\ \times,\ \div_p,\ \min,\ \max,\ \mathrm{neg} \}, \tag{11}$$

where neg is the unary sign change and $\div_p$ denotes protected division, defined as $a \div_p b = a/b$ if $|b| > 10^{-9}$ and a otherwise. The rule is deployed inside a semi-active schedule generation scheme (SGS): starting from the empty schedule, at each of the $n\,m$ decision points the set $\mathcal{E}$ of eligible operations (those whose job predecessor is already scheduled) is formed, the tree is evaluated on the attributes of every $o \in \mathcal{E}$, and the operation with *lowest* priority value is appended at its earliest feasible start on its machine, Eq. (3).

Because the lowest value is dispatched, a criterion that seeks to *maximize* an attribute must be expressed through the unary neg operator. Classic rules are thus points of this search space: shortest processing time is the single-node tree PT, whereas most work remaining is neg(WKR). We inject both as seeded rules in the initial population.

Priority ties at dispatch time are broken by eligible set order (lowest job index first), which is deterministic; hence a rule maps each instance to exactly one schedule.

4.2. Evolutionary components

The initial population is built with the *ramped half-and-half* method [22, 23]: each individual draws its maximum depth uniformly from $\{2, 3, 4\}$ and is generated, with equal probability, by the *full* method, which expands every branch with function nodes down to that depth, or by the *grow* method, which may close any branch earlier by placing a terminal (probability 0.3 per node in our implementation). In both cases internal nodes are drawn uniformly from $\mathcal{F}$ and leaves uniformly from the terminal set. The last two members of the population are not generated at random: they are the two

Algorithm 1 GP hyper-heuristic for the IJSP

$P \leftarrow$ ramped half-and-half($pop - 2$) $\cup$ {SPT, MWKR}
evaluate(P) ▷ mean RE of one constructive pass on the training instances
for $g = 1$ **to** *gens* **do**
    $P' \leftarrow$ elite(P, e) ▷ elitism
    while $|P'| < pop$ **do**
        if rand() $< p_c$ **then**
            $p_1, p_2 \leftarrow$ tournament(P, k), tournament(P, k)
            child $\leftarrow$ subtreeCrossover(p_1, p_2)
        else
            child $\leftarrow$ subtreeMutation(tournament(P, k))
        end if
        $P' \leftarrow P' \cup \{$*child*$\}$ ▷ fitness ∞ if size $> c$
    end while
    $P \leftarrow$ evaluate(P')
end for
return best rule in P

seeded rules of Section 4.1, inserted directly. Every individual is evaluated by the mean RE of its deterministic constructive pass over the four training instances, computed with the same environment, decoder and evaluator used by all baselines.

Each generation then proceeds as follows. Parents are chosen by tournament, drawing k individuals uniformly without replacement from the current population and returning the fittest of them. With probability p_c two parents are recombined by *subtree crossover* [22, 23]: a node is drawn uniformly

Table 2: Evolution parameters: the configuration selected by irace (Section 5.3), used for every experiment reported in this paper.

Population / generations	100 / 50
Initialization	ramped half-and-half (depths 2–4)
Seeded individuals	PT and neg(WKR)
Selection	tournament, size $k = 7$
Crossover / mutation prob.	$p_c = 0.77$ / 0.23 (subtree, depth ≤ 3)
Elitism	$e = 2$
Tree-size cap	$c = 30$ nodes (violations: fitness ∞)
Fitness	mean RE, deterministic rollout, TA11–TA14
Seeds	1–30 (RNG of evolution and environment)

at random in each parent, leaves included, and the offspring is a copy of the first parent with the subtree rooted at its chosen node replaced by the one rooted at the second parent's. Unlike the classical two-offspring formulation, we keep a single offspring per recombination. With the complementary probability $1 - p_c$ a single parent undergoes *subtree mutation* [22, 23]: a node is again drawn uniformly at random, and the subtree rooted at it is discarded and replaced by a freshly generated random tree, built with the grow method and maximum depth 3 over the full primitive set. Offspring whose size exceeds a cap of c nodes receive infinite fitness and are thereby excluded from selection. The depth bounds above apply only to trees created from scratch, at initialization and inside mutation; during the run only the node count is bounded, so recombination may produce deeper expressions.

Replacement is fully generational, the offspring becoming the next population, except for the e best individuals of the current generation, which are copied into it unchanged. The process stops after a fixed number of

generations and returns the best rule found. Algorithm 1 summarizes the generational loop and Table 2 lists the parameter values.

4.3. Randomized multi-start extension

A deterministic rule produces one schedule. For matched comparisons with sampling-based methods we wrap the evolved rule in GRASP-style randomization [38]: with probability 0.1 the dispatched operation is drawn uniformly from the eligible set, otherwise the rule decides; sample 0 remains deterministic. The wrapper, which we call GP-best-of-N, reports the lexicographically best of its N schedules. This uniform perturbation of a single rule is the simplest member of a family of randomized rule-based builders; an alternative swaps among a *set* of evolved rules during construction [39].

5. Experimental Setup

5.1. Benchmark instances

We use two benchmarks, both taken from the IJSP literature [2, 3, 4]: the 70 interval versions of Taillard's instances [36] (TA1–TA70, seven size classes from 15×15 to 50×20), and the 12 classical instances on which the published IJSP metaheuristics report results (FT10, FT20 [40], seven instances of the La family [41] and ABZ7–9 [20], from 10 to 20 jobs and 5 to 15 machines). Both were obtained from their crisp counterparts with the generation scheme introduced in [2]: for every operation a value δ is drawn uniformly at random in $[0,\, 0.15\,p]$, where p is its original duration, and that duration becomes the integer interval $[p-\delta,\, p+\delta]$, machine routes being unchanged. The interval is symmetric around p, so the crisp instance is exactly the midpoint scenario of its interval version. Since rules are evolved on Taillard instances only and applied to the classical set without any re-evolution, the latter doubles as

a cross-family generalization test (Section 6.4). Both sets are released with this paper (see Data availability).

5.2. Training protocol, data split and baselines

The 70 interval Taillard instances are split three ways. Rules are evolved on TA11–TA14 (20×15). TA15–TA20 form the development set: they never contribute to fitness, but the design and configuration decisions of this paper, including the irace study of Section 5.3, were selected on their performance. The remaining 60 instances are unseen by every design and configuration decision and, together with the 12 classical instances, constitute the generalization test.

Each evolution runs a population of 100 rules for 50 generations and returns the best rule of its final population; thirty independent evolutions, differing only in their random seed, yield thirty rules. All thirty are evaluated on the 70 Taillard instances, and the one with the lowest mean RE is the *featured rule*. This selection does look at the 60 held-out Taillard instances, which is why every table reports the mean over the 30 rules beside the featured rule's value; the 12 classical instances play no part in any choice.

5.3. Hyperparameter configuration with irace

Four parameters of the evolution were configured with irace [42]: tournament size, crossover probability, tree-size cap and elitism. Population size and number of generations were held fixed, so that every configuration is evaluated under the same evolution budget. The budget was 200 experiments, each one a full evolution (population 100, 50 generations) on the training instances, scored by the resulting rule's RE on the development set. Table 3 lists the parameter space and the winning configuration (Table 2). Of the

Table 3: Hyperparameter space explored by irace (200 experiments; cost = development-set RE of the evolved rule) and the winning configuration.

Parameter	Range / values	irace winner
Tournament size	$\{2, 3, 4, 5, 7\}$	7
Crossover prob. p_c	[0.5, 0.95]	0.77
Tree-size cap	$\{20, 30, 40, 60\}$	30
Elitism	$\{1, 2, 4\}$	2

four elite configurations irace retains, three favor a high selection pressure (tournament size 7) and all four fall in a narrow crossover band, 0.72–0.77; the tree-size cap does not discriminate, the elites spanning caps of 20, 30 and 40.

6. Results

6.1. Evolution behavior and seed stability

Over 30 independent evolutions, training RE converges to 14.9 ± 1.5 (range 12.4–19.7), with most of the improvement realized by generation 25–30 (Figure 2). Figure 3 shows the featured rule, obtained with seed 1: 26 nodes over 12 leaves. Every terminal of the tree is non-negative by construction and processing times are at least one time unit, so both guards it contains resolve to the same branch at every decision. On the problem domain the tree is therefore exactly equivalent to the expression

$$SLACK^2 \; + \; 2\,PT \; - \; WKR \; - \; WKRW \; - \; 1. \tag{12}$$

We verified this identity at each of the 37,250 dispatching decisions the rule takes on the benchmark (1.29 million candidate evaluations). Since the

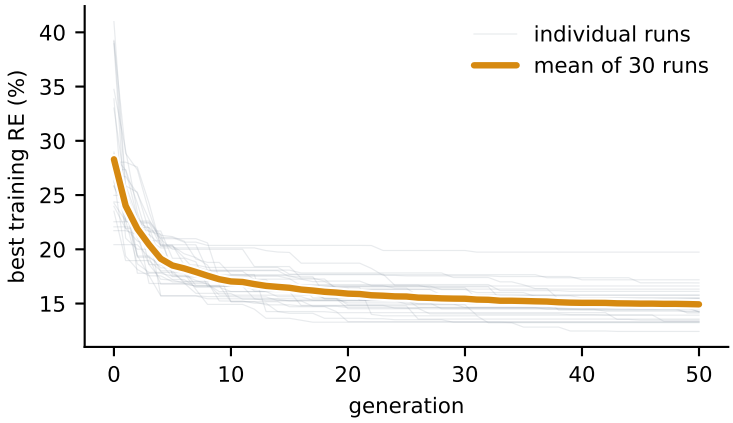

Figure 2: Best training fitness per generation over the 30 independent evolutions: each individual run and their mean.

lowest value is dispatched, the rule reads directly: $-WKR$ is most-work-remaining, $2\,PT$ is shortest-processing-time at double weight, $SLACK^2$ penalizes quadratically any operation that cannot start at the earliest available time, and the constant is immaterial to the ranking. The remaining term, $-WKRW$, is the interval-aware one: among jobs with comparable remaining work the rule advances those whose remaining work is *more uncertain*, resolving that uncertainty early rather than carrying it to the end of the schedule. The evolved rule is thus a weighted blend of MWKR and SPT, regularized by a quadratic earliest-start term and tilted by the uncertainty of the work ahead.

6.2. Generalization to the full benchmark

Table 4 reports single-rollout RE per size class over all 70 instances, as mean ± standard deviation across the 30 evolved rules and for the single best rule. The best rule attains 17.71% global mean. Evolved only on 20×15, its best class is the unseen 50×15 (13.8%), and the same pattern holds on

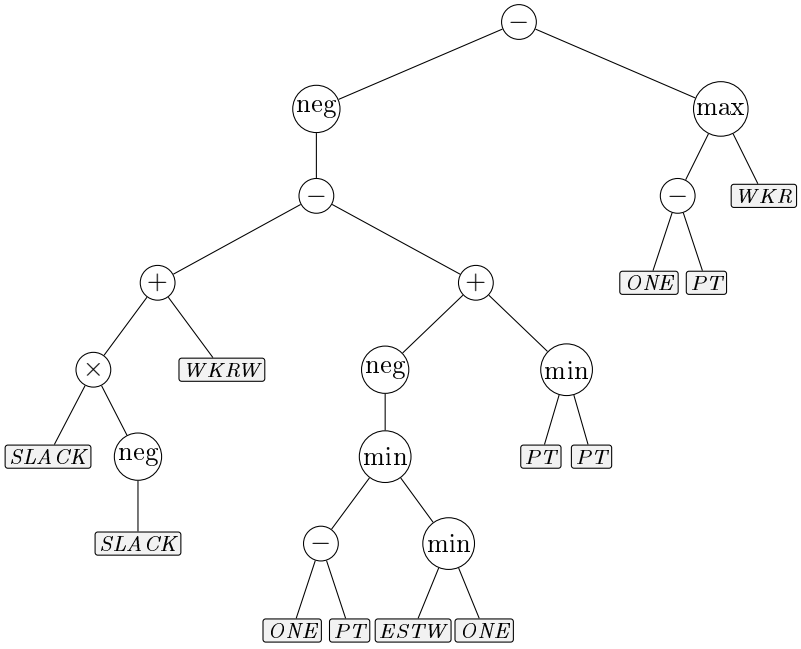

Figure 3: The featured rule (seed 1), which attains 17.71% RE over the 70 instances. Circles are functions and shaded boxes the interval-aware terminals of Table 1; the operation with the lowest value of the expression is dispatched. The tree is shown exactly as evolved; Eq. (12) gives its simplified form.

Table 4: RE (%) per size class, single deterministic rollout, over all 70 Taillard interval instances: mean ± sd across the 30 independently evolved rules, and the best evolved rule.

	15×15	20×15	20×20	30×15	30×20	50×15	50×20	All
Mean of 30	18.1±1.3	18.0±1.5	20.9±1.7	21.4±1.7	25.7±1.9	13.7±1.5	15.1±1.5	18.99±1.33
Best rule	15.7	16.6	18.1	21.1	24.1	**13.8**	14.6	**17.71**

average (13.7 ± 1.5). Because all terminals are per-operation attributes with no size-bound quantities, transfer requires no retraining or rescaling.

6.3. Position among constructive methods

Table 5 reports every constructive baseline under the protocol of this paper, one deterministic pass per instance with the same decoder and evaluator for all methods, together with a uniformly random dispatcher as a reference point. Since these rules are deterministic, the standard deviation is taken across instances and measures how uniform a method is over the benchmark rather than any run-to-run variability; the random dispatcher, the one stochastic entry, is averaged over 10 dispatches per instance. Times are wall-clock averages over the same 70 instances with 5 repetitions each; being implementation-bound, they are informative only in comparison between rows. The two GP entries are the mean over the 30 evolved rules and the best of them, the mean being what a single evolution should be expected to yield.

Two observations are needed to read the table correctly. First, the plain rules that ignore machine availability (SPT, LPT and the critical ratio CR, the latter being a due-date rule applied here without due dates) perform *worse than random* in this setting. This is a property of the schedule generation scheme, not a defect of the criteria: because a solution is decoded as a permutation whose operations are appended to their machines, systematically dispatching by processing time alone produces highly unbalanced permutations. The same criterion embedded in the Giffler–Thompson conflict set, which restores classical dispatching semantics, behaves as expected (SPT: 627% as a plain rule versus 70.6% under G&T). Second, the criteria that do account for the state of the shop (EST, MOR, MWKR) are better than random, and the strongest baseline overall is G&T-MWKR at 29.5%, which is the reference we use throughout.

Against that strongest baseline, the evolved rules average 18.99 ± 1.33

Table 5: Constructive baselines on the 70 interval Taillard instances: mean RE (%), standard deviation across instances, and mean time in seconds of one constructive pass.

Method	RE (%)	sd	Time (s)
LPT	703.6	168.0	0.23
SPT	627.2	152.2	0.23
CR	625.7	140.6	0.26
Random dispatcher	*127.2*	*13.9*	*0.09*
G&T-SPT	70.6	10.0	0.29
MWKR	64.7	10.4	0.33
MOR	45.5	10.1	0.33
EST	42.3	10.6	0.28
G&T-MWKR	**29.5**	6.4	0.38
GP rule (mean of 30)	18.99	4.96	0.34
GP rule (best of 30)	**17.71**	5.23	0.34

RE over the 30 evolutions, and the best rule attains 17.71%.

The gap holds at the level of individual instances: the best evolved rule outperforms each of the eight deterministic baselines on every one of the 70 instances, and a paired Wilcoxon signed-rank test rejects equality against each of them at $p < 10^{-6}$ ($z = -7.3$); the conclusion is unchanged under Holm's correction for the eight comparisons. The narrowest margin over the whole comparison is 3.9 points, against G&T-MWKR on a single 20×15 instance; against the best plain rule (EST) the smallest margin is 6.3 points. Per-instance figures are given in Table A.1 (appendix). This quality comes at no evaluation penalty over hand-crafted rules: evaluating the evolved expression tree over the eligible set costs what evaluating a hand-crafted criterion costs. Averaged over the 70 instances, a complete constructive pass

takes 0.34 s with the evolved rule against 0.33 s with MOR and 0.38 s with G&T-MWKR, a spread of 14% between the three; all of them reduce to one vectorized pass over the eligible operations per decision, so the choice of priority expression contributes little to the cost of a pass. Both cost and quality thus place the evolved rule with the constructive methods, while closing much of the quality gap that separated them from search-based methods; that comparison is deferred to Section 6.4, where published results are available on a common benchmark.

6.4. Cross-family generalization: the 12 classical instances

The 12 classical instances serve a double purpose: they test the evolved rule on instance families and sizes it has never seen, since it was evolved on four interval Taillard 20×15 instances and is applied here without any re-evolution, and they enable a direct comparison with the *published* results of the IJSP metaheuristics on their own benchmark and metric (Table 6). Those methods belong to different algorithmic families, evolutionary computation and swarm intelligence: a genetic algorithm evolving a population of permutations [2] and three artificial bee colony variants [3, 4], each of which runs a separate search on every instance. Alongside the deterministic pass, Table 6 reports the randomized extension of Section 4.3 at $N = 64$ and $N = 1024$: independent randomized passes of the same rule, of which the best schedule is kept, with no refinement or recombination among samples.

The two families also differ in how they spend computation: GP-1024 evaluates 1024 mutually independent samples per instance, whereas the metaheuristics evolve a population of 250 individuals [2, 3] through successive generations, each of which depends on the previous one. In wall-clock terms, a single constructive pass over these instances takes 0.03–0.13 s in our Python

Table 6: RE (%) on the 12 classical interval instances: our methods (deterministic single pass and GP-best-of-N; rule evolved on interval Taillard 20×15, applied zero-shot) versus the published average results (30 runs per instance) of the IJSP metaheuristics GA [2], ABC_{E3}, fEABC [3] and ESABC [4]. LB is the best-known lower bound of the expected makespan.

		Ours (zero-shot)			Published (avg of 30)			
Inst.	LB	GP	GP-64	GP-1024	GA	ABC_{E3}	fEABC	ESABC
FT10	930	20.4	8.4	6.6	5.2	4.1	3.5	3.0
FT20	1165	13.2	12.1	10.8	4.4	1.7	1.8	1.8
La21	1046	15.0	14.4	9.0	5.0	5.0	4.2	4.0
La24	935	10.1	8.8	8.6	6.3	5.1	4.9	5.0
La25	977	11.8	11.6	7.6	5.1	3.9	3.4	2.7
La27	1235	25.0	13.7	11.3	10.2	4.7	4.6	4.1
La29	1152	13.4	14.2	11.0	14.2	8.6	7.4	7.0
La38	1196	15.0	13.3	9.4	9.2	6.9	6.1	5.8
La40	1222	13.3	12.8	9.9	8.7	4.2	4.6	4.1
ABZ7	656	19.7	14.3	11.7	12.5	7.3	6.6	6.7
ABZ8	645	24.7	22.1	17.7	18.5	12.1	11.1	10.9
ABZ9	661	33.4	25.3	22.1	18.0	13.1	11.6	11.2
Mean		17.9	14.2	11.3	9.8	6.4	5.8	5.5

implementation, 0.07 s on average, against the 0.5–2.2 s reported for the genetic algorithm and the 1.9–6.8 s reported for fEABC on the same instances [3], both implemented in C++. GP-1024, however, requires 37–134 s of sequential Python time depending on instance size, and is therefore *not* faster than the published metaheuristics.

Transfer is a property of the method rather than of one rule: over the 30 evolved rules the deterministic pass averages $19.04 \pm 1.97\%$ RE on these in-

stances, single rules ranging from 15.3% to 23.3%. Since the two benchmarks use different reference bounds (Section 3), RE values do not compare across them, but margins over baselines evaluated on the same instances do, and that margin is preserved: 2.40 times over MOR and 1.58 over G&T-MWKR on the classical set, against 2.40 and 1.55 on the benchmark the rules were evolved for.

Against the per-instance metaheuristics the evolved rules do not compete: the purpose-built ABC variants remain clearly ahead (5.5–6.4%), as expected of iterative search, and even wrapped as GP-1024 the featured rule reaches 11.3% against the genetic algorithm's published 9.8%, better on 3 of the 12 instances (paired Wilcoxon $z = 1.96$, $p \approx 0.05$). What the comparison locates is the constructive frontier: rules evolved once on another benchmark and applied here with no retraining average 19.04% in a single pass, against 30.1% (G&T-MWKR) and 45.8% (MOR) for the deterministic baselines, and the best rule with a 1024-sample budget comes within 1.5 points of a population-based method that searches every instance separately.

7. Analysis

This section takes up the three questions of Section 1 in turn: which attributes the evolution selects (Section 7.1); whether the evolution driven by the makespan objective extracts measurable value from the width terminals (Section 7.2); and whether an objective that penalizes interval width yields more robust schedules, and at what cost (Sections 7.3 and 7.4).

7.1. Rule anatomy and interpretability

The output of GP is a readable expression. Evolved rules are compact, with a mean tree size of 26 nodes (20–30) and mean depth 9, so a rule can

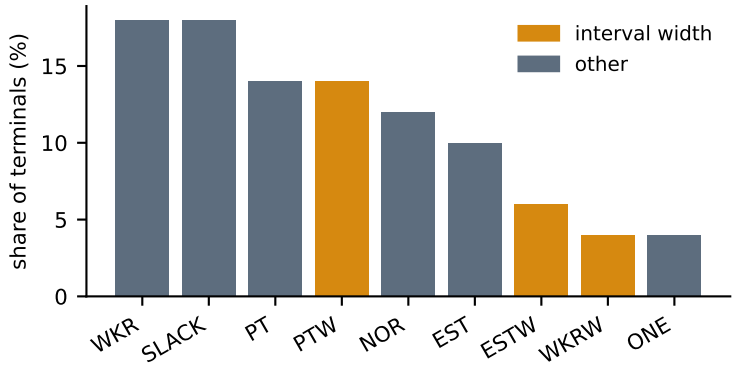

Figure 4: Terminal usage across the 30 evolved rules, as a share of all terminals. Interval-width terminals (amber) expose the magnitude of the uncertainty to the rule.

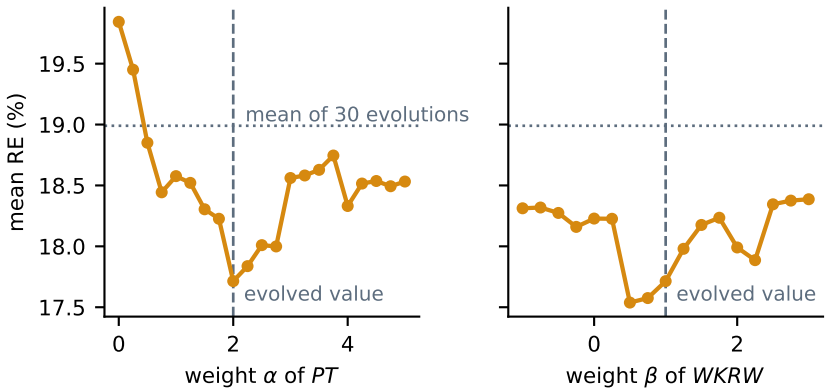

Figure 5: Sensitivity of the featured rule to the two weights of its simplified form, Eq. (12): mean RE over the 70 instances when each weight is varied and the other is held at its evolved value. Dashed lines mark the evolved weights, the dotted line the mean over the 30 evolved rules. $\beta = 0$ is the rule with its interval-width term switched off.

be inspected and simplified by hand. Figure 4 reports how often each terminal is used across the 30 evolved rules. The most frequent are the classical scheduling attributes, slack ($SLACK$), work remaining (WKR) and earli-

est start (EST), together with worst-case processing time (PT), indicating that the evolution rediscovers known dispatching logic rather than exploiting benchmark artifacts. The *interval-width* terminals (PTW, $ESTW$, $WKRW$) account for ≈21% of all terminal usage and appear in 27 of the 30 evolved rules. Whether that width information translates into better schedules is quantified in Sections 7.2–7.4.

The simplified form of the featured rule, Eq. (12), also admits a direct sensitivity analysis: it weighs PT by $\alpha = 2$ and $WKRW$ by $\beta = 1$, and we swept each weight over the 70 instances holding the other at its evolved value (Figure 5). Neither curve is unimodal: over the sampled grid α has five local minima and β three, and since each point is a deterministic evaluation over the 70 instances, that ruggedness belongs to the landscape rather than to sampling noise. The evolved $\alpha = 2$ is nonetheless the global minimum of its sweep; the evolved β is not, giving 17.71% against the 17.54% of $\beta = 0.5$. A post-evolution weight sweep is thus a cheap refinement for any simplified rule. The multimodality matters if the sweep is replaced by local descent: started at $\alpha = 4$, a descent stalls in a secondary minimum 0.6 points above the global one. Setting $\beta = 0$ removes the width term from the deployed rule and costs 0.5 points of RE (18.23 against 17.71).

7.2. The value of interval awareness: ablations

The interval-width terminals are the distinctive ingredient of this setting, so we isolate their contribution by ablation: we repeat the 30 evolutions with and without the width terminals (PTW, $ESTW$, $WKRW$) in the terminal set, everything else held fixed, and compare the resulting rules with a paired Wilcoxon signed-rank test on the shared seeds, reporting the normal-approximation statistic z, exact two-sided p-values, and the matched-pairs

Table 7: Ablations of the interval machinery (30 independent evolutions per arm; paired rank tests between arms): the terminal ablation under two fitness objectives, and the crisp-midpoint control, evolved with scalar fitness and no interval information, tested against the interval-trained arm above it. *Width* is the relative width of the predicted makespan interval.

Fitness objective	Terminal set	RE (%)	Width (%)
Makespan $E[\mathbf{C}_{\max}]$	full	18.99 ± 1.33	12.41 ± 0.30
	without widths	18.66 ± 1.08	12.62 ± 0.23
	test	*n.s.* ($z = -0.83$)	$z = 3.40$, $p < 0.001$
Midpoint control	without widths	18.63 ± 0.99	12.55 ± 0.17
	test	*n.s.* ($z = -0.13$)	*n.s.* ($z = -1.29$)
Robust, Eq. (13)	full	19.62 ± 1.52	12.04 ± 0.75
	without widths	18.42 ± 0.87	12.47 ± 0.20
	test	$z = -3.75$, $p < 0.001$	$z = 3.22$, $p < 0.001$

rank-biserial correlation $|r|$ as effect size. Holm's correction across the tests of Table 7, and across each later family of comparisons, changes no verdict reported in this paper. The irace configuration of Section 5.3 was selected on the full terminal set and is used for both arms: holding it fixed is what makes the comparison controlled, at the cost that the ablated arm runs with hyperparameters tuned for a larger primitive set than its own. Table 7 collects the outcome.

Under the makespan objective, the width terminals do not improve the quantity being optimized. Removing the three of them leaves the expected makespan unchanged: 18.66 ± 1.08 mean RE over the 70 instances without them versus 18.99 ± 1.33 with them, a difference that is not significant ($z = -0.83$) and, if anything, favors the smaller terminal set. They do, however,

affect the schedules: the relative width of the predicted makespan interval is significantly narrower when the rule can read widths (12.41 ± 0.30 versus 12.62 ± 0.23; paired Wilcoxon $z = 3.40$, $p < 0.001$, $|r| = 0.71$), even though nothing in this objective rewards narrowness. The terminals appear in 27 of the 30 rules (Section 7.1); their effect under this objective is a slightly tighter interval at no cost in expected makespan.

Interval information enters the evolution at two points: through the terminal set, which fixes what a rule can read, and through the fitness, which scores every candidate schedule by interval arithmetic. The ablation above removes the first but keeps the second: rules evolved without width terminals are still selected by the interval-evaluated expected makespan.

To remove the second as well, we evolve 30 further rules on the crisp counterparts of the training instances, exactly their midpoint scenarios (Section 5.1), with the same seeds, configuration and reduced terminal set: these evolutions never see an interval, and their fitness is the plain scalar makespan. Deployed unchanged on the interval benchmark, they are indistinguishable from the arm evolved with interval fitness, 18.63 ± 0.99 against 18.66 ± 1.08 RE ($z = -0.13$, n.s.), with predicted widths equally close (12.55 ± 0.17 against 12.62 ± 0.23, $z = -1.29$, n.s.), although the two fitness functions are not identical, since max does not commute with taking midpoints. For expected makespan, then, neither component is needed: rules evolved with no interval information at all match the interval-trained arm. The objective of the next section penalizes the width of the makespan interval and cannot be replicated on a crisp instance, where that width is identically zero.

7.3. The robust objective: trading expected makespan for predictability

The makespan fitness gives the evolution no incentive to trade expected makespan for predictability. To test whether it can make that trade when rewarded for it, we repeat the terminal ablation with a *robust* fitness that explicitly rewards predictability, minimizing

$$f_\lambda = \overline{C}_{\max} + \lambda \left(\overline{C}_{\max} - \underline{C}_{\max} \right), \tag{13}$$

that is, a low worst case *and* a narrow interval, with the weight λ setting the balance between the two; we take $\lambda = 1$ here. Under this objective the separation widens. Rules evolved with the width terminals produce schedules whose makespan interval is significantly narrower than those evolved without them (12.04 ± 0.75 versus 12.47 ± 0.20 relative width; paired Wilcoxon $z = 3.22$, $p < 0.001$, $|r| = 0.67$), and the gain is paid for in expected makespan (19.62 versus 18.42 RE), the trade the robust objective asks for.

The weight λ of Eq. (13) governs that trade-off, and sweeping it traces the frontier between quality and predictability (Figure 6). Both quantities move monotonically with λ: as the weight grows from 0.5 to 4, the relative width of the makespan interval falls from 12.18 ± 0.50 to 10.54 ± 1.26 while the expected makespan degrades from 19.20 ± 1.52 to 23.27 ± 3.80 RE. At the far end of the sweep the schedules are therefore narrower than any the ablated rules produced, at a cost of about four points of RE. The practitioner can choose an operating point along this curve.

Sweeping λ for the ablated arm (ten evolutions per value) shows that it has no such curve. Its width does not respond to the weight at all: 12.47 ± 0.16 at $\lambda = 0$, 12.43 ± 0.15 at 0.5, 12.47 ± 0.20 at 1, 12.44 ± 0.24 at 2 and 12.56 ± 0.17 at 4, a range of 0.13 points against the 1.64 the full arm covers, with the largest width at the largest weight, and with a spread several times

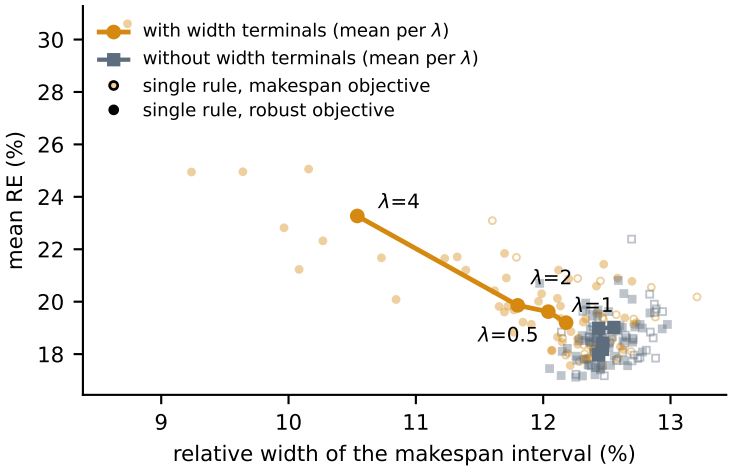

Figure 6: Quality–predictability plane over the 70 instances. Faint markers are individual evolved rules: amber with the interval-width terminals, grey without them; open markers for the makespan objective, filled for the robust one at any λ. Solid curves join the mean of each arm at each λ of the robust-fitness sweep, Eq. (13). With the width terminals, increasing λ trades expected makespan for narrower intervals; without them the five means, $\lambda = 0$ to 4, fall within 0.13 points of width and are left unlabeled.

smaller, so no individual rule escapes either. Penalizing width in the fitness therefore buys narrowness only when the rule has terminals through which to act on it: the objective and the representation are not two independent routes to the same end, the first works through the second.

The rule-level layer of Figure 6 shows the same separation rule by rule. Without the width terminals no rule goes below 12.05% under either objective; with them, the minimum is 11.60% under the makespan objective and 9.24% under the robust one. The terminals alone therefore suffice to leave the ablated band, but reaching 9.24% takes both them and an objective that rewards narrowness.

Table 8: Predicted quality and executional robustness over the 70 instances (lower is better on all measures): mean RE, relative width of the predicted makespan interval, and $\bar{\varepsilon}$ ($\times 10^3$) at nominal interval widths and widened by +20% and +40% (Monte Carlo, K=1000 realizations, common random numbers across methods). Each GP arm is represented by one rule: the makespan arms by their best RE, the robust arms by their narrowest predicted interval, which is what their fitness optimizes.

			$\bar{\varepsilon}$ ($\times 10^3$)		
Method	RE (%)	Width (%)	+0%	+20%	+40%
GP, makespan	17.71	12.28	5.70	6.64	7.57
GP, makespan, no widths	17.17	12.48	5.75	6.80	7.83
GP, robust λ=1	24.95	9.24	4.52	5.32	6.08
GP, robust λ=1, no widths	17.45	12.05	5.54	6.50	7.42
GP, robust λ=4	28.27	8.61	4.21	5.01	5.73
G&T-MWKR	29.50	13.77	6.29	7.51	8.58
EST	42.30	13.37	5.66	6.79	7.83

7.4. Robustness under executional uncertainty

Relative error measures a-priori quality; the $\bar{\varepsilon}$ robustness of Eq. (10) measures how well that a-priori prediction survives execution, and is the quantity examined here. We estimate it by Monte Carlo, drawing K=1000 realizations of the durations uniformly on each operation's interval and executing the fixed processing order on each of them; lower is more robust. To probe behavior under growing uncertainty we also widen every interval symmetrically about its midpoint by +20% and +40% (clipping negative bounds to zero), re-evaluating the same processing orders.

Under the makespan objective the first question receives the same negative answer (Table 8): the featured rule and its ablated counterpart are

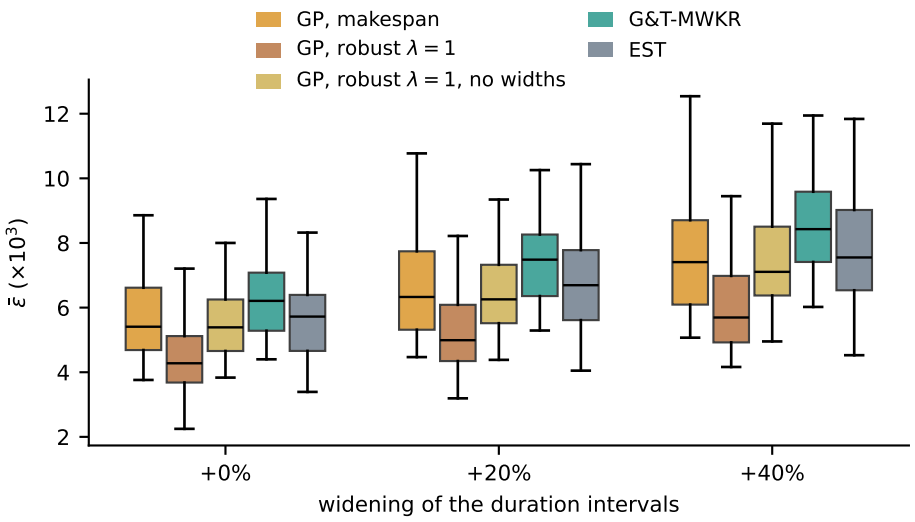

Figure 7: Per-instance $\bar{\varepsilon}$ by method, with the duration intervals at their nominal widths and widened by +20% and +40% (lower is better). The robust λ=1 entries are the narrowest rule of that arm and its counterpart evolved without the width terminals, as in Table 8.

statistically indistinguishable in $\bar{\varepsilon}$ (5.70 against 5.75 at the nominal width; paired Wilcoxon over the 70 instances, $z = -0.17$). The rule is more robust than G&T-MWKR (6.29, $z = -4.42$, $p < 0.001$, $|r| = 0.61$) but not than EST (5.66, $z = 0.42$), although it improves on EST by 24.6 points of RE. A low deviation does not require a low makespan, so robustness must be optimized for directly.

The robust fitness achieves this. The narrowest rule of the λ=1 arm reduces $\bar{\varepsilon}$ from 5.70 to 4.52 ($z = -6.40$, $|r| = 0.88$ against the makespan rule), and the reduction requires the width terminals: under the same fitness, its ablated counterpart reaches 5.54 ($z = -6.25$, $|r| = 0.86$). At λ=4 the deviation falls to 4.21. The narrower predicted intervals of Section 7.3 thus

correspond to smaller realized deviations, at the cost in expected makespan reported in the RE column: 24.95 and 28.27 against 17.71.

The comparisons above select one representative rule per arm; at the level of the method the ordering is the same. Recomputing $\bar{\varepsilon}$ at the nominal width for all 30 rules of each arm and pairing by seed, the makespan arm averages 5.67 ± 0.16 against 5.77 ± 0.14 for its ablation ($z = -2.58$, $p < 0.05$ after Holm's correction over the three arm-level tests, $|r| = 0.54$), the robust λ=1 arm 5.50 ± 0.30 against 5.68 ± 0.11 for its own ($z = -3.26$, $p < 0.01$, $|r| = 0.68$), and the robust arm improves on the makespan arm with the same terminals ($z = -2.46$, $p < 0.05$, $|r| = 0.51$). Two things follow. The width terminals do reduce $\bar{\varepsilon}$ slightly even under the makespan objective, a method-level effect too small to show in the single featured pair; and the arm-level reductions are modest, three percent or less, against the fifth separating the featured rules, so selecting the narrowest rule of the robust arm amplifies an effect that at the method level is consistent but small.

Figure 7 shows the per-instance $\bar{\varepsilon}$ distributions. The ordering is stable as uncertainty grows, and the advantage of the robust rule widens with it: at +40% its deviation is 6.08 against 7.57 for the makespan rule and 8.58 for G&T-MWKR.

7.5. Limitations

Four limitations bound the scope of our conclusions. First, the evolved rules choose from the whole eligible set inside a semi-active decoder. Neither the conflict-set restriction that makes Giffler and Thompson's procedure yield active schedules, used here only for the baselines, nor an insertion-based generation scheme was placed in the evolutionary loop, and both are known to help weaker constructors. Second, all rules were evolved on in-

stances generated with the ±15% symmetric interval scheme standard in the IJSP literature; the robustness analysis of Section 7.4 does re-evaluate the resulting schedules under intervals widened by +20% and +40%, but no rule was evolved under a different width, and asymmetric intervals remain untested. Third, our runtime figures are relative (evolved rules cost the same as hand-crafted ones per pass); absolute times depend on the research-grade Python implementation, but the rankings reported here do not. Finally, no learned policy is among the baselines: no published DRL method targets the IJSP [30].

8. Conclusions and Future Work

We presented what is, to the best of our knowledge, the first GP hyper-heuristic for job shop scheduling with interval durations: priority rules evolved over a terminal set that exposes both the worst-case values and the widths of the uncertain quantities, compact enough to be read and simplified by hand. The empirical conclusions are three. (i) Evolved rules outperform every deterministic constructive baseline on the 70-instance benchmark by a wide, statistically significant margin (18.99 ± 1.33 over 30 evolutions, 17.71% for the best rule, versus 29.5% for the strongest baseline, which the best rule outperforms on all 70 instances) at the cost of a single constructive pass, and (ii) they generalize zero-shot across all size classes, evolved on 20×15 and best on the unseen 50×15; since every terminal is a per-operation attribute with no size-bound quantities, that transfer requires no retraining or rescaling. (iii) For expected makespan, interval awareness at evolution time is unnecessary: removing the width terminals leaves it unchanged, and evolving on the crisp midpoint instances with scalar fitness

does equally well. Under an objective that penalizes interval width, however, an objective with no crisp counterpart, the predicted intervals become substantially narrower and the deviation of the executed makespan from its prediction falls by about a fifth (4.52 against 5.70×10^{-3}); that reduction requires the width terminals, since under the same objective the ablated arm reaches only 5.54 and no value of λ narrows its intervals. The gain is paid for in expected makespan, and the weight λ governs the trade-off.

Returning to the questions of Section 1: the evolution selects the classical scheduling attributes most often, with the width terminals present in most rules; the magnitude of the uncertainty adds nothing to the expected makespan, which rules reading bounds or midpoints alone attain equally well, and shows its value on the third question: an objective that rewards a narrow interval, which only the width terminals can pursue, pays a chosen amount of expected makespan for executions about a fifth closer to their prediction.

Future work follows four lines. First, optimizing the numeric weights of an evolved rule, which the sensitivity analysis of Section 7.1 suggests the evolution leaves unfinished. Two properties of this setting shape how such a search should be posed. Because operations are dispatched by *argmin*, the priority is invariant to adding a constant and to multiplying by a positive one, so the standard technique of linear scaling [43] cannot help and only the *relative* weights of the additive terms are free, $k - 1$ of them for a rule with k terms. And because the terminal set contains no ephemeral random constants, the evolution must build coefficients structurally, spending nodes of a bounded budget on arithmetic that a constant would express in one. Both an enriched terminal set and a local search over the relative weights, whether applied after evolution or to the elites of each generation, follow from these observations. Second, richer symbolic models: ensembles of comple-

mentary rules [27] and co-evolution of construction and tie-breaking. Third, using evolved rules as inexpensive, high-quality population initializers for the leading IJSP metaheuristics: the pools of diverse sequences produced by GP-best-of-N are directly usable as seeded initial populations. Fourth, a controlled comparison with learned neural constructive policies under matched training and inference budgets, the head-to-head evaluation that the recent literature [30] identifies as missing.

CRediT authorship contribution statement

Hernán Díaz: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing – original draft, Writing – review & editing, Visualization, Project administration.

Acknowledgements

This work was supported by the Spanish Ministry of Science, Innovation and Universities (MCIN/AEI/10.13039/501100011033) under grant PID2022-141746OB-I00.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The GP hyper-heuristic is implemented from scratch in pure Python/NumPy, with no evolutionary-computation dependencies; evolved

rules are stored as plain JSON expression trees and evaluated as drop-in dispatching strategies by the same environment used by every baseline. All benchmark instances, the 220 evolved rules of every experimental arm, the primary result files behind each table and figure, and a self-contained implementation that re-derives the deposited results from scratch are openly available in a Zenodo repository [44] (doi:10.5281/zenodo.21716972).

References

[1] A. Allahverdi, A survey of scheduling problems with uncertain interval/bounded processing/setup times, Journal of Project Management 7 (4) (2022) 255–264. doi:10.5267/j.jpm.2022.3.003.

[2] H. Díaz, I. González-Rodríguez, J. J. Palacios, I. Díaz, C. R. Vela, A genetic approach to the job shop scheduling problem with interval uncertainty, in: Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU), Springer, 2020, pp. 663–676. doi:10.1007/978-3-030-50143-3_52.

[3] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Fast elitist ABC for makespan optimisation in interval JSP, Natural Computing 22 (4) (2023) 645–657. doi:10.1007/s11047-023-09953-2.

[4] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, An elitist seasonal artificial bee colony algorithm for the interval job shop, Integrated Computer-Aided Engineering 30 (3) (2023) 223–242. doi:10.3233/ica-230705.

[5] J. Branke, S. Nguyen, C. W. Pickardt, M. Zhang, Automated design of production scheduling heuristics: a review, IEEE Transactions on

Evolutionary Computation 20 (1) (2016) 110–124. doi:10.1109/tevc.2015.2429314.

[6] S. Nguyen, Y. Mei, M. Zhang, Genetic programming for production scheduling: a survey with a unified framework, Complex & Intelligent Systems 3 (1) (2017) 41–66. doi:10.1007/s40747-017-0036-x.

[7] J. J. Palacios, I. González-Rodríguez, C. R. Vela, J. Puente, Coevolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop, Fuzzy Sets and Systems 278 (2015) 81–97. doi:10.1016/j.fss.2014.12.003.

[8] A. Allahverdi, H. Aydilek, A. Aydilek, Single machine scheduling problem with interval processing times to minimize mean weighted completion time, Computers & Operations Research 51 (2014) 200–207. doi:10.1016/j.cor.2014.06.003.

[9] W. Xu, W. Wu, Y. Wang, Y. He, Z. Lei, Flexible job-shop scheduling method based on interval grey processing time, Applied Intelligence 53 (2023) 14876–14891. doi:10.1007/s10489-022-04213-9.

[10] P. Zheng, S. Xiao, P. Zhang, Y. Lv, A two-individual-based evolutionary algorithm for the flexible assembly job shop scheduling problem with uncertain interval processing times, Applied Sciences 14 (22) (2024) 10304. doi:10.3390/app142210304.

[11] S. Afşar, J. Puente, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Solving the energy-aware flexible job shop problem using a memetic algorithm, Integrated Computer-Aided Engineering 32 (4) (2025) 397–414. doi:10.1177/10692509251353343.

[12] Y. N. Sotskov, T.-C. Lai, F. Werner, Measures of problem uncertainty for scheduling with interval processing times, OR Spectrum 35 (3) (2013) 659–689. doi:10.1007/s00291-012-0306-3.

[13] Y. N. Sotskov, N. M. Matsveichuk, V. D. Hatsura, Schedule execution for two-machine job-shop to minimize makespan with uncertain processing times, Mathematics 8 (8) (2020) 1314. doi:10.3390/math8081314.

[14] D. Lei, Population-based neighborhood search for job shop scheduling with interval processing time, Computers & Industrial Engineering 61 (4) (2011) 1200–1208. doi:10.1016/j.cie.2011.07.010.

[15] H. Díaz, J. J. Palacios, I. Díaz, C. R. Vela, I. González-Rodríguez, Robust schedules for tardiness optimization in job shop with interval uncertainty, Logic Journal of the IGPL 31 (2) (2022) 240–254. doi:10.1093/jigpal/jzac016.

[16] S. S. Panwalkar, W. Iskander, A survey of scheduling rules, Operations Research 25 (1) (1977) 45–61. doi:10.1287/opre.25.1.45.

[17] R. Haupt, A survey of priority rule-based scheduling, OR Spektrum 11 (1) (1989) 3–16. doi:10.1007/bf01721162.

[18] M. Đurašević, D. Jakobović, A survey of dispatching rules for the dynamic unrelated machines environment, Expert Systems with Applications 113 (2018) 555–569. doi:10.1016/j.eswa.2018.06.053.

[19] B. Giffler, G. L. Thompson, Algorithms for solving production-scheduling problems, Operations Research 8 (4) (1960) 487–503. doi:10.1287/opre.8.4.487.

[20] J. Adams, E. Balas, D. Zawack, The shifting bottleneck procedure for job shop scheduling, Management Science 34 (3) (1988) 391–401. doi:10.1287/mnsc.34.3.391.

[21] J. Carlier, The one-machine sequencing problem, European Journal of Operational Research 11 (1) (1982) 42–47. doi:10.1016/s0377-2217(82)80007-6.

[22] J. R. Koza, Genetic Programming: On the Programming of Computers by Means of Natural Selection, MIT Press, Cambridge, MA, 1992.

[23] R. Poli, W. B. Langdon, N. F. McPhee, A Field Guide to Genetic Programming, Lulu Enterprises, 2008, freely available at http://www.gp-field-guide.org.uk.

[24] T. Hildebrandt, J. Branke, On using surrogates with genetic programming, Evolutionary Computation 23 (3) (2015) 343–367. doi:10.1162/evco_a_00133.

[25] Y. Mei, S. Nguyen, B. Xue, M. Zhang, An efficient feature selection algorithm for evolving job shop scheduling rules with genetic programming, IEEE Transactions on Emerging Topics in Computational Intelligence 1 (5) (2017) 339–353. doi:10.1109/TETCI.2017.2743758.

[26] S. Nguyen, M. Zhang, M. Johnston, K. C. Tan, Automatic design of scheduling policies for dynamic multi-objective job shop scheduling via cooperative coevolution genetic programming, IEEE Transactions on Evolutionary Computation 18 (2) (2014) 193–208. doi:10.1109/TEVC.2013.2248159.

[27] F. J. Gil-Gala, C. Mencía, M. R. Sierra, R. Varela, Learning ensembles of priority rules for online scheduling by hybrid evolutionary algorithms, Integrated Computer-Aided Engineering 28 (1) (2021) 65–80. doi:10.3233/ica-200634.

[28] F. J. Gil-Gala, M. Đurašević, M. R. Sierra, R. Varela, Tree search hyper-heuristic with application to combinatorial optimization, Journal of Heuristics 31 (2025) 25. doi:10.1007/s10732-025-09560-7.

[29] D. Karunakaran, Y. Mei, G. Chen, M. Zhang, Dynamic job shop scheduling under uncertainty using genetic programming, in: Intelligent and Evolutionary Systems, Springer, 2017, pp. 195–210. doi:10.1007/978-3-319-49049-6_14.

[30] M. Xu, Y. Mei, F. Zhang, M. Zhang, Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning, Artificial Intelligence Review 58 (2025) 160. doi:10.1007/s10462-024-11059-9.

[31] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, C. Xu, Learning to dispatch for job shop scheduling via deep reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS), 2020.

[32] J. Park, J. Chun, S. H. Kim, Y. Kim, J. Park, Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning, International Journal of Production Research 59 (11) (2021) 3360–3377. doi:10.1080/00207543.2020.1870013.

[33] R. E. Moore, R. B. Kearfott, M. J. Cloud, Introduction to Interval Analysis, SIAM, Philadelphia, PA, 2009. doi:10.1137/1.9780898717716.

[34] A. Sprecher, R. Kolisch, A. Drexl, Semi-active, active, and non-delay schedules for the resource-constrained project scheduling problem, European Journal of Operational Research 80 (1) (1995) 94–102. doi:10.1016/0377-2217(93)e0294-8.

[35] H. Ishibuchi, H. Tanaka, Multiobjective programming in optimization of the interval objective function, European Journal of Operational Research 48 (2) (1990) 219–225. doi:10.1016/0377-2217(90)90375-L.

[36] E. Taillard, Benchmarks for basic scheduling problems, European Journal of Operational Research 64 (2) (1993) 278–285. doi:10.1016/0377-2217(93)90182-M.

[37] V. J. Leon, S. D. Wu, R. H. Storer, Robustness measures and robust scheduling for job shops, IIE Transactions 26 (5) (1994) 32–43. doi:10.1080/07408179408966626.

[38] T. A. Feo, M. G. C. Resende, Greedy randomized adaptive search procedures, Journal of Global Optimization 6 (2) (1995) 109–133. doi:10.1007/BF01096763.

[39] F. J. Gil-Gala, M. Đurašević, M. R. Sierra, J. Puente, Greedy randomised adaptive solution builder using priority rules, in: Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '25), ACM, 2025, pp. 211–214. doi:10.1145/3712255.3726547.

[40] H. Fisher, G. L. Thompson, Probabilistic learning combinations of local job-shop scheduling rules, in: J. F. Muth, G. L. Thompson (Eds.), Industrial Scheduling, Prentice-Hall, Englewood Cliffs, NJ, 1963, pp. 225–251.

[41] S. Lawrence, Resource constrained project scheduling: an experimental investigation of heuristic scheduling techniques (supplement), Tech. rep., Graduate School of Industrial Administration, Carnegie-Mellon University, Pittsburgh, PA (1984).

[42] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, T. Stützle, The irace package: Iterated racing for automatic algorithm configuration, Operations Research Perspectives 3 (2016) 43–58. doi:10.1016/j.orp.2016.09.002.

[43] M. Keijzer, Improving symbolic regression with interval arithmetic and linear scaling, in: Genetic Programming (EuroGP), Springer, 2003, pp. 70–82. doi:10.1007/3-540-36599-0_7.

[44] H. Díaz, Genetic programming dispatching rules for the interval job shop: instances, evolved rules, results and code, [dataset] (2026). doi:10.5281/zenodo.21716972.

Appendix A. Per-instance results

Table A.1 lists, for each of the 70 interval Taillard instances, the crisp lower bound and the single-pass RE (%) of the evolved rule and of the two strongest baselines: EST, the best of the plain dispatching rules, and G&T-MWKR, the best baseline overall.

Table A.1: Per-instance RE (%) of the evolved rule (GP) and of the two strongest baselines, EST and G&T-MWKR (single deterministic pass). LB is the crisp Taillard lower bound.

Inst.	LB	EST	G&T	GP	Inst.	LB	EST	G&T	GP
TA1	1231	49.9	26.1	19.3	TA36	1819	41.1	42.8	23.8
TA2	1244	24.9	27.8	14.2	TA37	1771	43.4	35.6	18.4
TA3	1218	33.0	33.2	14.1	TA38	1673	45.0	28.0	20.1
TA4	1175	32.0	27.7	13.0	TA39	1795	46.6	28.9	22.4
TA5	1224	27.2	33.0	14.7	TA40	1651	46.9	42.3	26.1
TA6	1238	26.8	18.2	13.7	TA41	1906	44.6	45.0	33.4
TA7	1227	30.7	26.1	17.7	TA42	1884	71.1	38.1	24.3
TA8	1217	23.7	25.0	14.8	TA43	1809	68.7	39.8	22.3
TA9	1274	37.7	28.2	20.0	TA44	1948	52.1	34.7	25.8
TA10	1241	37.0	22.1	15.4	TA45	1997	50.2	33.7	14.9
TA11	1357	56.2	30.7	17.5	TA46	1957	60.8	31.6	22.4
TA12	1367	48.8	34.6	17.6	TA47	1807	46.8	30.1	25.6
TA13	1342	60.2	26.9	13.3	TA48	1912	59.1	33.7	21.1
TA14	1345	39.5	29.7	13.2	TA49	1931	51.0	35.3	20.1
TA15	1339	48.1	30.6	16.7	TA50	1833	61.4	46.2	31.3
TA16	1360	45.2	34.0	15.6	TA51	2760	33.8	34.7	23.2
TA17	1462	58.9	17.7	13.9	TA52	2756	38.0	26.5	12.7
TA18	1377	43.8	31.8	23.6	TA53	2717	34.1	18.3	11.8
TA19	1332	45.8	29.3	20.0	TA54	2839	20.9	16.1	8.2
TA20	1348	45.7	24.1	15.1	TA55	2679	31.7	29.2	15.5
TA21	1642	33.4	28.0	20.2	TA56	2781	34.8	19.8	10.9
TA22	1561	44.2	30.8	17.4	TA57	2943	32.9	20.0	9.6
TA23	1518	42.8	35.3	17.8	TA58	2885	37.4	26.3	13.6
TA24	1644	56.3	23.8	14.1	TA59	2655	26.6	28.2	16.7
TA25	1558	45.8	33.8	16.6	TA60	2723	41.7	19.8	15.5
TA26	1591	42.8	31.2	26.9	TA61	2868	45.7	25.4	15.4
TA27	1652	50.0	31.1	21.4	TA62	2869	43.0	27.6	17.2
TA28	1603	36.8	26.8	12.4	TA63	2755	38.8	23.8	13.4
TA29	1583	28.8	32.0	17.2	TA64	2702	38.2	28.4	12.2
TA30	1528	34.2	36.1	17.0	TA65	2725	42.9	30.0	20.5
TA31	1764	30.7	34.4	24.5	TA66	2845	41.8	28.0	11.9
TA32	1774	42.7	33.3	19.0	TA67	2825	31.7	26.1	15.2
TA33	1788	51.3	34.8	27.5	TA68	2784	50.1	19.1	10.6
TA34	1828	45.9	31.8	20.7	TA69	3071	35.3	22.8	12.9
TA35	2007	30.3	24.1	8.4	TA70	2995	42.9	23.4	16.7